

Affinity Propagation (AP)

Структурированный разбор метода **Affinity Propagation (AP)** с примерами кода на Python, объяснением математики и задачами для проверки модели.

Суть метода (Affinity Propagation)

Affinity Propagation (AP) — это алгоритм кластеризации, который не требует заранее задавать количество кластеров. Он работает путем обмена сообщениями между точками данных до сходимости.

Основные понятия:

Экземпляр (Exemplar) — точка данных, которая представляет кластер.

Сходство (Similarity) $s(i,k)$ — обычно отрицательное евклидово расстояние: $s(i,k) = -\|x_i - x_k\|^2$.

Ответственность (Responsibility) $r(i,k)$ — насколько точка k подходит быть экземпляром для точки i .

Доступность (Availability) $a(i,k)$ — насколько точка k "доступна" для точки i в качестве экземпляра.

Математические формулы

Обновление ответственности: $r(i,k) \leftarrow s(i,k) - \max_{k' \neq k} [a(i,k') + s(i,k')]$

Обновление доступности: $a(i,k) \leftarrow \min [0, r(k,k) + \sum_{i' \notin \{i,k\}} r(i',k)]$

Для диагонали ($i=k$): $a(k, k) \leftarrow \sum_{i' \neq k} \max (0, r(i',k))$

Демпфирование (чтобы избежать осцилляций):

$$r_{t+1}(i,k) = \lambda \cdot r_t(i,k) + (1-\lambda) \cdot r_{t+1}(i,k)$$

$$a_{t+1}(i,k) = \lambda \cdot a_t(i,k) + (1-\lambda) \cdot a_{t+1}(i,k)$$

Реализация на Python (scikit-learn)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import AffinityPropagation
from sklearn.datasets import make_blobs

# 1. Генерация данных
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.9, random_state=0)

# 2. Создание и обучение модели
```

```
af = AffinityPropagation(damping=0.9, preference=-50, random_state=0)
```

```
af.fit(X)
```

```
# 3. Результаты
```

```
labels = af.labels_
```

```
cluster_centers = af.cluster_centers_indices_
```

```
n_clusters = len(cluster_centers)
```

```
print(f'Число кластеров: {n_clusters}')
```

```
print(f'Индексы экземпляров: {cluster_centers}')
```

```
# 4. Визуализация
```

```
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', s=50)
```

```
plt.scatter(X[cluster_centers, 0], X[cluster_centers, 1], c='red', s=200, marker='X',  
label='Экземпляры')
```

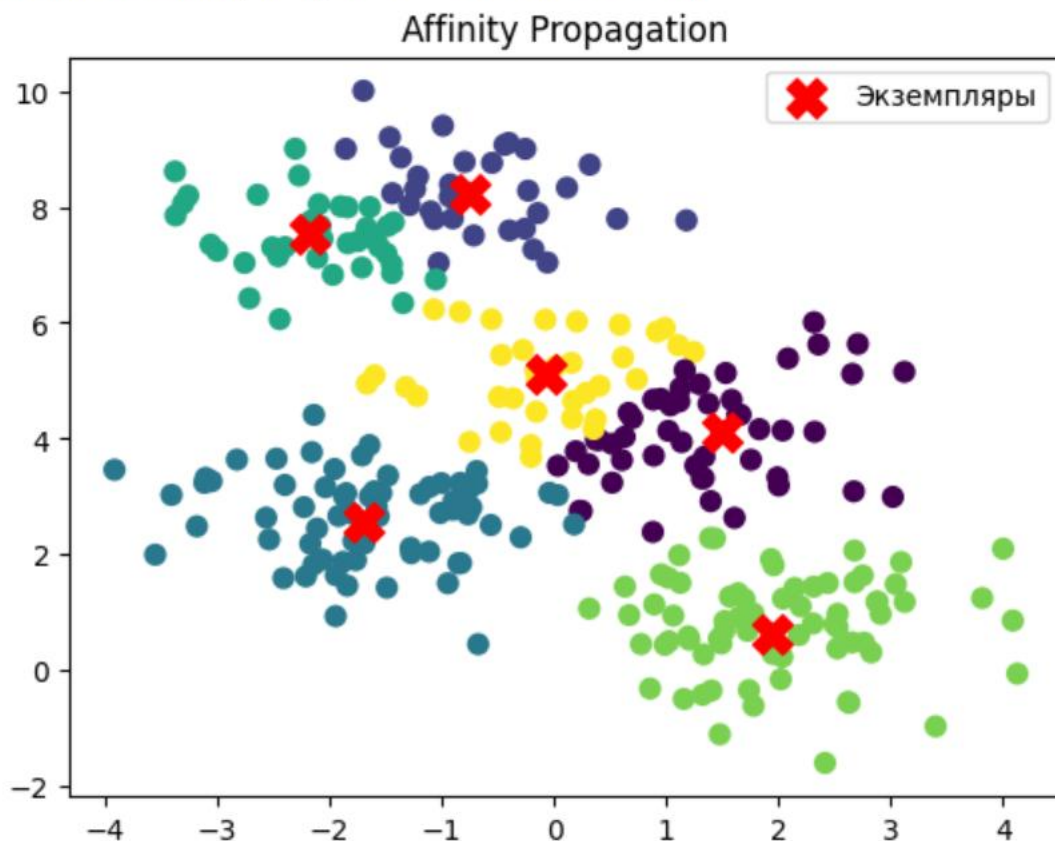
```
plt.title('Affinity Propagation')
```

```
plt.legend()
```

```
plt.show()
```

```
... Число кластеров: 6
```

```
Индексы экземпляров: [ 70 156 211 220 236 241]
```



Параметры модели:

Параметр	Описание	Влияние
Damping	Коэффициент затухания (обычно 0,5-1)	Стабилизирует сходимость
Preference	“Предпочтение” каждой точке быть экземпляром	Чем выше, тем больше кластеров
Affinity	Метод вычисления сходства ('euclidean' или 'precomputed')	Зависит от данных

Задачи для проверки понимания

Задача 1. Влияние preference

Вопрос: Как изменится число кластеров, если увеличить preference?

Ответ: Увеличение preference делает каждую точку более "склонной" стать экземпляром → число кластеров **увеличится**.

Задача 2. Влияние damping

Вопрос: Зачем нужен damping и что произойдет, если он слишком мал?

Ответ: Damping сглаживает обновления, чтобы избежать осцилляций. Если слишком мал — алгоритм может не сойтись (колебания).

Задача 3. Сложность алгоритма

Вопрос: Почему AP плохо работает на больших данных ($N > 10\,000$)?

Ответ: Временная сложность $O(N^2T)$ и память $O(N^2)$ делают его непрактичным для больших N из-за квадратичной зависимости.

Задача 4. Программная: подбор параметров

Условие: Дан набор данных из 500 точек с 5 реальными кластерами, напишите код для подбора preference так, чтобы число найденных кластеров было близко к 5.

```
from sklearn.metrics import silhouette_score
```

```
best_pref = None
```

```
best_score = -1
```

```
best_n = 0
```

```

for pref in np.linspace(-100, -10, 10):
    af = AffinityPropagation(damping=0.9, preference=pref, random_state=0)
    labels = af.fit_predict(X)
    n_clusters = len(np.unique(labels))
    # Игнорируем случай 1 кластера
    if n_clusters > 1:
        score = silhouette_score(X, labels)
        if abs(n_clusters - 5) < abs(best_n - 5) or (abs(n_clusters - 5) == abs(best_n -
5) and score > best_score):
            best_pref = pref
            best_score = score
            best_n = n_clusters
print(f'Лучший preference: {best_pref}, кластеров: {best_n}, Silhouette:
{best_score:.3f}')

```

Лучший preference: -80.0, кластеров: 4, Silhouette: 0.536

Итог: Affinity Propagation — мощный метод для задач, где количество кластеров неизвестно.

Плюсы: Автоматическое определение числа кластеров. Не требует случайной инициализации.

Минусы: Высокая вычислительная сложность.

Чувствителен к параметрам preference и damping.